WELT 4 · DOPPELSTUNDEN 24 UND 25 (KÜR) · IDE-LEVEL 4.1 UND 4.2

W4·1 – Der Hafen: Stapel und Schlange

Datei: `AQ_W4_L4_Hafen_Geruest.txt` (Klasse Hafen: Level 4.1 · Zugbuch und Initiative: Level 4.2) · Im Lab: **Level 4.1** und **Level 4.2** · Teil 1 in DS 24, Teil 2 in DS 25 (Kür)

Hinter dem Labyrinth öffnet sich der **Hafen**. Am Kai stapeln die Kräne Kisten zu Türmen, vor der Mole warten die Schiffe in einer langen Reihe auf ihren Liegeplatz, und die Hafenmeisterin führt eine Ladeliste, die ständig wächst und schrumpft. Wer hier arbeitet, muss genau wissen, **welche** Kiste er als Nächstes greifen kann und **welches** Schiff als Nächstes dran ist.

Für euer Dungeon-Spiel baut ihr daraus Spielmodule: **Rückgängig/Wiederholen** mit Stapeln und die **Zugreihenfolge** im Kampf mit einer Schlange. Im Abitur kommen Stapel und Schlange in fast jedem Jahrgang vor – meist als Befehlsfolge, deren Zustände ihr skizzieren müsst, und als Methode, die ohne Laufzeitfehler arbeiten muss.

Teil 1 – Stapel und Schlange (DS 24)

Aufgabe 1: Teller und Kasse

PAPIER ★

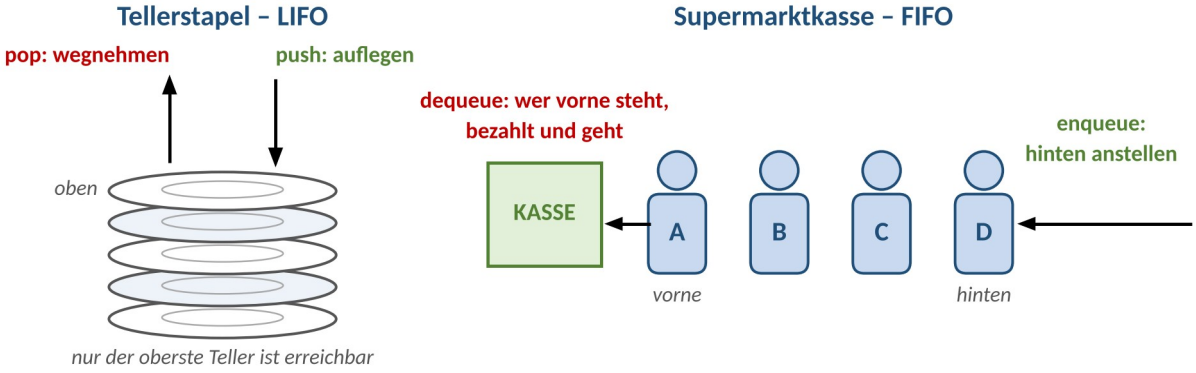


Abbildung 1: Zwei Ordnungen aus dem Alltag

a) **Tellerstapel:** Du darfst Teller nur **oben** auflegen oder wegnehmen und hast eine zweite, leere Ablagefläche. Beschreibe, wie du an den untersten von fünf Tellern kommst, sodass die übrigen vier danach wieder in derselben Reihenfolge auf ihrem Platz stehen. Wie viele Teller-Bewegungen brauchst du? **[2 BE]**

b) **Supermarktkasse:** A, B und C stehen an (A vorne). Erst bezahlt jemand, dann stellt sich D an, dann bezahlt wieder jemand. Wer hat bezahlt, wer steht jetzt vorne? Wie sähe eine „Stapel-Kasse“ aus – und warum wäre sie unfair? **[2 BE]**

c) Nenne je ein Beispiel aus der Informatik, in dem nach dem Prinzip „zuletzt hinein, zuerst heraus“ (LIFO) bzw. „zuerst hinein, zuerst heraus“ (FIFO) gearbeitet wird. **[1 BE]**

Infoteil 1: Abstrakte Datentypen – Schnittstelle und Implementierung

Ob ein Tellerstapel aus Porzellan oder aus Pappe ist, ist egal – wichtig ist nur, **was man mit ihm tun darf**. Genau das beschreibt ein **abstrakter Datentyp** (ADT):

- **Schnittstelle** („Bedienfeld"): welche Operationen es gibt und was sie bewirken. Für das Abitur sind die Schnittstellen von Stapel, Schlange und dynamischer Reihung in den Ergänzenden Hinweisen (EH § 4) festgelegt.
- **Implementierung** („Motorraum"): wie die Struktur innen gebaut ist – z. B. mit einer Reihung oder einer Kette aus Knoten (Expertenpfad in DS 29). Die Implementierung kann ausgetauscht werden, ohne dass sich ein Programm ändert, das nur die Schnittstelle benutzt.
- **Inhaltstyp:** was gespeichert wird. In Signaturen schreibst du z. B. „Stapel vom Inhaltstyp Kiste" oder „Schlange vom Inhaltstyp Ganzzahl".

Merke: Auf Stapel und Schlange greifst du **nur** über ihre Operationen zu – kein Index, kein Blick in die Mitte. Was nicht oben bzw. vorne liegt, ist erst nach dem Entnehmen der anderen Elemente erreichbar.

Infoteil 2: Stapel und Schlange nach EH § 4

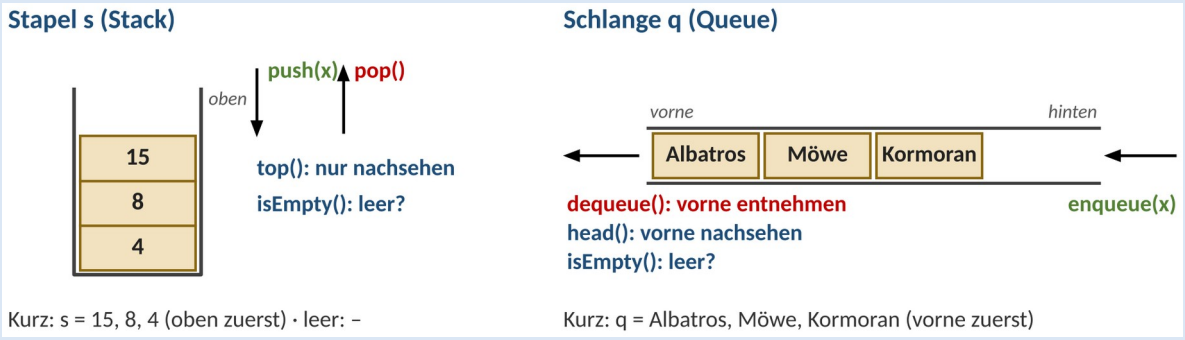


Abbildung 2: Stapel als Turm, Schlange als Reihe – und die Kurzschreibweise

Stapel (Stack) – LIFO	Wirkung
c Stack()	erzeugt einen leeren Stapel
isEmpty(): Wahrheitswert	wahr, wenn der Stapel kein Element enthält
push(x: Inhaltstyp): nichts	legt x oben auf den Stapel
pop(): Inhaltstyp	entfernt das oberste Element und liefert es – **nur bei nicht leerem Stapel**
top(): Inhaltstyp	liefert das oberste Element, **ohne** es zu entfernen – nur bei nicht leerem Stapel

Schlange (Queue) – FIFO	Wirkung
c Queue()	erzeugt eine leere Schlange
isEmpty(): Wahrheitswert	wahr, wenn die Schlange kein Element enthält
enqueue(x: Inhaltstyp): nichts	stellt x hinten an
dequeue(): Inhaltstyp	entfernt das vorderste Element und liefert es – **nur bei nicht leerer Schlange**
head(): Inhaltstyp	liefert das vorderste Element, **ohne** es zu entfernen – nur bei nicht leerer Schlange

Zustände skizzieren (wie im Abitur): Stapel als Turm mit markiertem „oben“ oder kurz **von oben nach unten**: s = 15, 8, 4. Schlange als Reihe mit „vorne“ und „hinten“ oder kurz **von vorne nach hinten**. Eine leere Struktur kennzeichnest du mit –.

Infoteil 3: Die Abitur-Klassen in der Online-IDE

Stack, Queue und DynArray stehen erst zur Verfügung, wenn du die Bibliothek „Niedersachsen“ einschaltest – **einmal pro Workspace**:

1. Workspace in der Liste links mit der **rechten** Maustaste anklicken → „Einstellungen“.
2. Häkchen bei **„Niedersachsen“** setzen → OK. Jetzt kennt die IDE Stack, Queue, DynArray und BinTree.
3. Die gleichnamigen Java-Klassen (java.util.Stack, ArrayList, LinkedList …) sind dann ausgeblendet. Code aus dem Netz mit `Stack<Integer>` kompiliert deshalb nicht – das ist gewollt.

Casten bei Abitur-Klassen

Die Klassen der Bibliothek sind **nicht generisch**: Sie speichern alles als `Object`. Was du herausholst, wandelst du in den richtigen Typ um (**Cast**):

```
Stack s = new Stack();                  // ohne spitze Klammern!
Kiste k = (Kiste) s.pop();              // Stapel vom Inhaltstyp Kiste
int x = (Integer) q.dequeue();          // Schlange vom Inhaltstyp Ganzzahl
String n = (String) q.head();           // Schlange vom Inhaltstyp Zeichenkette
char c = (Character) s.pop();           // Stapel vom Inhaltstyp Zeichen
int g = ((Kiste) s.top()).getGewicht(); // erst casten, dann Methode aufrufen
```

- **Hineinlegen ohne Cast:** `s.push(5)`, `q.enqueue("Möwe")`, `s.push(k)` – Zahlen und Zeichen werden automatisch verpackt. `null` darf nicht hinein.
- **Fehlt** ein Cast, meldet schon der Compiler einen Fehler. Ein **falscher** Cast (z. B. `(Integer)` auf eine Kiste) fällt erst beim Ausführen auf: ClassCastException.
- Auch auf Papier (Boss-Kampf, Abitur in Java) schreibst du die Casts mit. In Struktogrammen entfallen sie.

Abi-Check „Leer?“ – erst fragen, dann nehmen

`pop`, `top`, `dequeue` und `head` auf einer **leeren** Struktur führen zum Laufzeitfehler (IllegalStateException). Im Abitur müssen Laufzeitfehler durch **explizite Abfragen** ausgeschlossen werden (EH § 4):

```
if (!s.isEmpty()) {             // Leer? – erst fragen ...
    Kiste k = (Kiste) s.pop();  // ... dann nehmen
}
while (!q.isEmpty()) {          // jede Runde neu fragen
    String n = (String) q.dequeue();
}
```

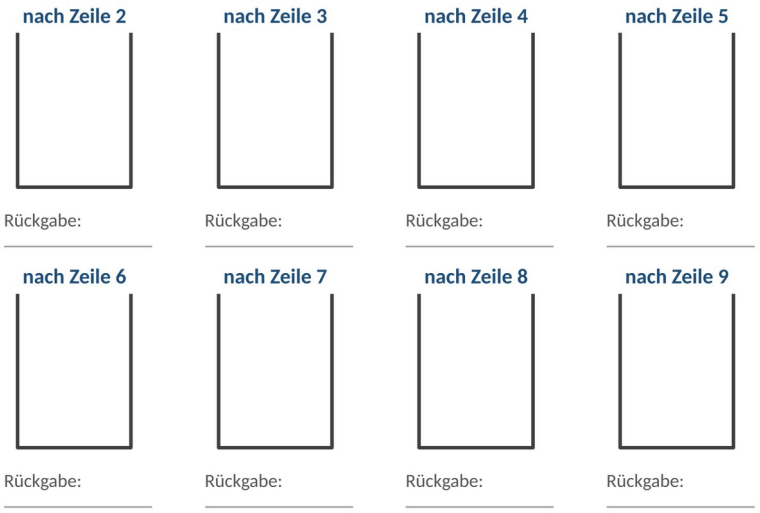
Aufgabe 2: Kistenturm und Schiffsschlange

PAPIER ★

Skizziere **nach jeder Zeile** den Inhalt der Struktur und notiere die Rückgabe (Anweisungen ohne Rückgabe: –).

a) Fülle die Türme von unten nach oben. **[3 BE]**

```
1  Stack s = new Stack();
2  s.isEmpty();
3  s.push(42);
4  s.push(137);
5  s.pop();
6  s.push(s.top());
7  s.push(18);
8  s.top();
9  s.isEmpty();
```



b) Fülle die Reihen von vorne (links) nach hinten. **[3 BE]**

```
1  Queue q = new Queue();
2  q.isEmpty();
3  q.enqueue(42);
4  q.enqueue(137);
5  q.enqueue(81);
6  q.head();
7  q.enqueue(q.dequeue());
8  q.dequeue();
```

vorne *hinten*

nach Zeile 2 Rückgabe: _____

nach Zeile 3 Rückgabe: _____

nach Zeile 4 Rückgabe: _____

nach Zeile 5 Rückgabe: _____

nach Zeile 6 Rückgabe: _____

nach Zeile 7 Rückgabe: _____

nach Zeile 8 Rückgabe: _____

c) In a) wird Zeile 3 gestrichen. Gib an, in welcher Zeile es dann zu einem Laufzeitfehler kommt, und begründe. **[1 BE]**

d) Schreibe Zeile 6 aus a) so um, dass sie nie zu einem Laufzeitfehler führt. **[1 BE]**

Aufgabe 3: Kräne im Wechselspiel – Mischaufgaben

PAPIER ★★

Ein Stapel s und eine Schlange q, beide vom Inhaltstyp Ganzzahl, haben den Startzustand s = 7, 2, 9 (oben zuerst) und q = 5, 1, 8 (vorne zuerst). Die Zeilen werden nacheinander ausgeführt.

```
1  s.push(q.dequeue());
2  s.push(s.pop());
3  q.enqueue(q.dequeue());
4  q.enqueue(s.pop());
5  int x = (Integer) s.pop() + (Integer) q.head();
6  s.push(x);
7  q.enqueue(s.top());
```

a) Gib nach jeder Zeile den Inhalt von q und s an, zu Zeile 5 auch den Wert von x. **[4 BE]**

Nr.	q (vorne → hinten)	s (oben → unten)	Bemerkung
0			
1			
2			
3			
4			
5			
6			
7			

Zeile 0 ist der Startzustand: q = 5, 1, 8 und s = 7, 2, 9.

b) Zeile 2 und Zeile 3 sehen ähnlich aus. Erläutere, warum Zeile 2 nichts verändert, Zeile 3 aber die Reihenfolge von q. **[2 BE]**

c) Erläutere, warum in Zeile 5 gecastet werden muss, in den Zeilen 1, 6 und 7 aber nicht. Was geschieht, wenn s vor Zeile 5 leer ist? **[2 BE]**

Aufgabe 4: Imbiss, Containerschiff, Rückgängig, Drucker

PAPIER ★★

Im Abitur heißt es oft: „Beurteilen Sie, welche Datenstruktur geeignet ist.“ Eine gute Begründung nennt das **Zugriffsprinzip** (LIFO, FIFO oder Zugriff über einen Index) und die **benötigten Operationen**.

a) Entscheide jeweils, ob ein Stapel, eine Schlange oder eine dynamische Reihung (Ausblick auf DS 27: Zugriff über einen Index, Einfügen und Löschen an jeder Stelle) am besten geeignet ist, und begründe. **[5 BE]**

Anwendung	Struktur	Begründung
(1) **Schnellimbiss:** Die Bedienung tippt jede Bestellnummer ein, die Küche bereitet die Bestellungen der Reihe nach zu.		
(2) **Containerschiff:** In einem Schacht stehen Container übereinander, der Kran greift immer den obersten.		
(3) **Texteditor:** Strg+Z nimmt die jeweils letzte Änderung zurück.		
(4) **Drucker:** Aufträge werden in der Reihenfolge ihres Eingangs gedruckt.		
(5) **Drucker im Lehrerzimmer:** Aufträge der Schulleitung werden vorgezogen, abgebrochene Aufträge aus der Mitte gelöscht.		

b) Die Nummern 3, 24, 5, 18, 10 kommen in dieser Reihenfolge an – als Container auf dem Schiff (Stapel) und als Bestellungen im Imbiss (Schlange). Skizziere beide Strukturen. **[2 BE]**

c) Aus beiden Strukturen soll die Folge 3, 24, 18, 6 werden (beim Stapel von unten nach oben gelesen). Gib jeweils eine Folge von Operationen mit möglichst wenigen Aufrufen von push/pop bzw. enqueue/dequeue an. Du darfst dir einzelne Werte in Variablen merken. Vergleiche die Anzahl der Aufrufe. **[3 BE]**

Aufgabe 5: Level 4.1 – Der Hafen

IDE ★ – ★★★

Arbeite in der Klasse **Hafen** (alle Methoden static), Testklasse `L41StapelSchlangeTest`. Zuerst die Bibliothek „Niedersachsen“ einschalten (Infoteil 3)!

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	stapleAuf(werte: Reihung von Ganzzahl): Stapel obersterWert(s: Stapel vom Inhaltstyp Ganzzahl): Ganzzahl leeren(s: Stapel): Ganzzahl	`basis_`
★★ Erweiterung	warteschlange(namen: Reihung von Zeichenkette): Schlange naechster(q: Schlange vom Inhaltstyp Zeichenkette): Zeichenkette alleAbfertigen(q: Schlange vom Inhaltstyp Zeichenkette): Zeichenkette	`erweiterung_`
★★★ Experte	umkehren(q: Schlange): nichts stapelUmdrehen(s: Stapel): Stapel	`experte_`

- Die Tests rufen jede Methode auch mit einer **leeren** Struktur auf. Beginnt eine Meldung mit „LAUFZEITFEHLER: … Leerabfrage mit isEmpty() vergessen?“, fehlt ein „Leer?“.
- Über jeder Methode im Gerüst steht, was bei leerer Struktur geliefert werden soll (z. B. −1 oder "").
- `obersterWert` soll nur **nachsehen** – der Test prüft, ob der Stapel danach noch genauso aussieht.
- `alleAbfertigen`: Das Trennzeichen ", " steht nur **zwischen** zwei Namen.
- Mit dem grünen Startknopf läuft das Hauptprogramm „Ein Tag im Hafen“ – vergleiche die Ausgaben mit deinen Skizzen.

> 🎯 **Im Lab üben**
>
> - **Level 4.1 „Kistenturm und Schiffsschlange: Stapel und Schlange“** – passt zu Teil 1, Aufgaben 1–4, und zu Aufgabe 9 (Bug-Hunt)
> - **Trainingsplatz Welt 4:** Zufallsaufgaben zu Stapel, Schlange und dynamischer Reihung (1 Punkt je Aufgabe)
>
> Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit deinem Namen und deinem persönlichen Code.

Teil 2 – Spielmodule: Rückgängig und Zugreihenfolge (DS 25, Kür)

Ein gutes Spiel verzeiht Fehler: Ein falscher Zug lässt sich **rückgängig** machen und – falls man es sich anders überlegt – **wiederholen**. Und im Kampf braucht es eine faire **Zugreihenfolge**: Wer gezogen hat, stellt sich hinten an. Beides baust du mit den Strukturen aus Teil 1.

Infoteil 4: Zwei Spielmodule

Zugbuch
- erledigt: Stapel vom Inhaltstyp Zeichenkette - zurueckgenommen: Stapel vom Inhaltstyp Zeichenkette
+ c Zugbuch() + zug(z: Zeichenkette): nichts + rueckgaengig(): Zeichenkette + wiederholen(): Zeichenkette + anzahlZuege(): Ganzzahl

Initiative
- reihe: Schlange vom Inhaltstyp Zeichenkette
+ c Initiative() + beitreten(name: Zeichenkette): nichts + amZug(): Zeichenkette + ausscheiden(name: Zeichenkette): Wahrheitswert

Abbildung 3: Klassenkarten Zugbuch und Initiative (Level 4.2)

- **Zugbuch** (Undo/Redo): Auf `erledigt` liegt der zuletzt ausgeführte Zug oben, auf `zurueckgenommen` der zuletzt zurückgenommene.
- `zug(z)` legt z auf erledigt. Nach einem neuen Zug kann nichts mehr wiederholt werden: Der Stapel zurueckgenommen wird **geleert**.
- `rueckgaengig()` nimmt den obersten Zug von erledigt, legt ihn auf zurueckgenommen und liefert ihn; ist nichts da, wird "" geliefert. `wiederholen()` arbeitet genau umgekehrt. `anzahlZuege()` liefert die Anzahl der Züge in erledigt – erledigt bleibt dabei unverändert.
- **Initiative:** In `reihe` steht vorne, wer als Nächstes am Zug ist. `beitreten(name)` stellt hinten an. `amZug()` entnimmt den Vordersten, stellt ihn **sofort hinten wieder an** (Rotieren) und liefert seinen Namen; leere Reihe → "".
- `ausscheiden(name)` entfernt das **erste** Vorkommen von name; die Reihenfolge der übrigen bleibt. Rückgabe wahr, wenn name dabei war.

Aufgabe 6: Das Zugbuch

PAPIER ★★

a) Ein neues Zugbuch erhält nacheinander die Aufrufe der Tabelle. Ergänze die Rückgaben und den Inhalt beider Stapel (oben zuerst). **[5 BE]**

Nr.	Aufruf	Rückgabe	erledigt	zurueckgenommen
1	`zug("Nord")`			
2	`zug("Ost")`			
3	`rueckgaengig()`			
4	`zug("Sued")`			
5	`zug("West")`			
6	`rueckgaengig()`			
7	`rueckgaengig()`			
8	`wiederholen()`			
9	`anzahlZuege()`			
10	`rueckgaengig()`			
11	`rueckgaengig()`			
12	`rueckgaengig()`			

b) Angenommen, zug leert den Stapel zurueckgenommen **nicht**. Gib den Inhalt von zurueckgenommen nach Zeile 8 an und erläutere, welcher Fehler beim übernächsten Aufruf von wiederholen() entstünde. **[2 BE]**

c) Ein Spiel soll nur die letzten **drei** Züge rückgängig machen können; ältere Züge sollen vergessen werden. Erläutere, warum das mit einem Stapel umständlich ist, und schlage eine besser geeignete Struktur vor. **[2 BE]**

Aufgabe 7: Initiative im Kampf

PAPIER ★★

a) Ein neues Initiative-Objekt erhält nacheinander die Aufrufe der Tabelle. Ergänze Rückgaben und den Inhalt von reihe (vorne zuerst). **[4 BE]**

Nr.	Aufruf	Rückgabe	reihe (vorne → hinten)
1	`beitreten("Aria")`		
2	`beitreten("Borin")`		
3	`beitreten("Cyra")`		
4	`beitreten("Goblin")`		
5	`amZug()`		
6	`amZug()`		
7	`amZug()`		
8	`ausscheiden("Aria")`		
9	`amZug()`		
10	`amZug()`		
11	`ausscheiden("Ork")`		
12	`amZug()`		

b) Ein Teammitglied schreibt ausscheiden wie unten. Gib für den Zustand nach Zeile 7 aus a) die Reihe nach `ausscheiden("Aria")` an. Vergleiche mit Zeile 8 aus a): Wer wäre jeweils als Nächster am Zug? **[2 BE]**

```
public boolean ausscheiden(String name) {
    while (!((String) reihe.head()).equals(name)) {
        reihe.enqueue(reihe.dequeue());          // rotieren, bis name vorne steht
    }
    reihe.dequeue();
    return true;
}
```

c) Nenne zwei weitere Situationen, in denen diese Fassung versagt, und beschreibe jeweils die Folge. **[2 BE]**

Aufgabe 8: Level 4.2 – Spielmodule

IDE ★ – ★★★

Arbeite in den Klassen **Zugbuch** und **Initiative** (Objekt-Klassen, Attribute vorgegeben), Testklasse `L42SpielmoduleTest`.

Stufe	Methoden	Tests beginnen mit
★ Basis	c Zugbuch(), zug, rueckgaengig c Initiative(), beitreten, amZug	`basis_`
★★ Erweiterung	wiederholen(): Zeichenkette anzahlZuege(): Ganzzahl (erledigt bleibt unverändert)	`erweiterung_`
★★★ Experte	ausscheiden(name: Zeichenkette): Wahrheitswert	`experte_`

- Die **Konstruktoren** schreibst du selbst. Fehlt ein `new Stack()` bzw. `new Queue()`, melden fast alle Tests „Zugriff auf null“.
- `anzahlZuege` darf erledigt nicht zerstören – überlege dir, wo du die Züge beim Zählen zwischenlagerst (Vorgriff auf DS 26).
- `ausscheiden`: Lerne aus Aufgabe 7 b) und c). Hängt der Testrunner, hast du eine Endlosschleife.

Aufgabe 9: Bug-Hunt im Hafenbüro

PAPIER ★★

Die Methode `obersteZwei(s: Stapel vom Inhaltstyp Kiste): Ganzzahl` soll das Gesamtgewicht der beiden obersten Kisten liefern. Liegt nur eine Kiste auf dem Stapel, wird ihr Gewicht geliefert, bei einem leeren Stapel 0. Danach muss s wieder genauso aussehen wie vorher. Die Klasse Kiste kennst du aus dem Gerüst (Abbildung 4).

```
1  public static int obersteZwei(Stack s) {
2      Kiste k1 = s.pop();
3      Kiste k2 = (Kiste) s.top();
4      int summe = k1.getGewicht() + k2.getGewicht();
5      s.push(k1);
6      return summe;
7  }
```

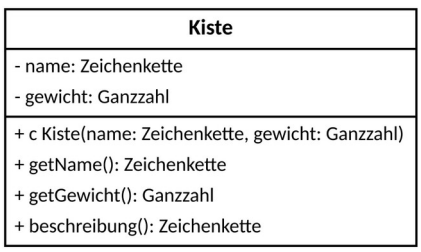


Abbildung 4: Klasse Kiste

a) Der Compiler meldet einen Fehler. Gib die Zeile und die Ursache an. **[1 BE]**

b) Nach der Korrektur des Compilerfehlers arbeitet die Methode für die Standard-Ladung (oben Wein 6 kg, darunter Tuch 8 kg) richtig. Gib für einen Stapel mit genau **einer** Kiste und für einen **leeren** Stapel an, in welcher Zeile ein Laufzeitfehler auftritt. **[2 BE]**

c) Schreibe eine korrigierte Fassung, die alle Fälle ohne Laufzeitfehler behandelt und s unverändert lässt. **[3 BE]**

d) Jemand ersetzt Zeile 3 durch `int g2 = (Integer) s.top();`. Erläutere, warum der Compiler das akzeptiert, und was beim Ausführen passiert. **[2 BE]**

Für Schnelle: Palindrome und Kranfolgen

- **Palindrom-Detektor:** Lege die Zeichen eines Wortes gleichzeitig auf einen Stapel und in eine Schlange und entnimm dann immer ein Zeichen aus beiden. Erkläre, warum du so Palindrome wie „RENTNER“ erkennst, und schreibe `istPalindrom(w: Zeichenkette): Wahrheitswert` mit Stack und Queue (an den (Character)-Cast denken!).
- **Kranfolgen:** Die Kisten 1, 2, 3 kommen in dieser Reihenfolge am Kran an. Der Kran darf jederzeit die nächste ankommende Kiste auflegen (push) oder die oberste abgeben (pop). Welche Reihenfolgen der abgegebenen Kisten sind möglich? Ist 3, 1, 2 möglich? Wie viele Reihenfolgen gibt es bei vier Kisten?

🎯 Im Lab üben

- **Level 4.2 „Kür: Zugbuch und Initiative – Spielmodule“** – passt zu Teil 2, Aufgaben 6–8

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit deinem Namen und deinem persönlichen Code.

Abi-Check Stapel und Schlange

- ☐ Zustände skizzieren: Stapel oben zuerst, Schlange vorne zuerst, leer „–“ – und angeben, wo oben bzw. vorne ist.
- ☐ Vor **jedem** pop/top bzw. dequeue/head: **Leer?** – mit isEmpty() fragen.
- ☐ top und head lesen nur, pop und dequeue entnehmen.
- ☐ Beim Entnehmen casten: (Kiste), (Integer), (String), (Character); Inhalte über Getter, Zeichenketten mit equals vergleichen.
- ☐ Eignung immer mit Zugriffsprinzip (LIFO, FIFO, Index) **und** benötigten Operationen begründen.